الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء العاشر: مكتبة Scikit-learn

الفصول ٧ - ١١: الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
الفصل السابع: الأمثلة العملية 4
٧.١ المستوى المبتدئ 4
مثال ١: خط أنابيب تصنيف بسيط من البداية للنهاية 4
٧.٢ المستوى المتوسط 4
مثال ٢: خط أنابيب موحّد بـ Pipeline بدل خطوات منفصلة 4
٧.٣ المستوى الاحترافي 4
مثال ٣: Pipeline كامل ببيانات مختلطة وبحث شبكي عن أفضل معاملات 4
٧.٤ مثال من مشروع حقيقي: تقييم عدة نماذج ومقارنتها بمقياس موحّد 5
تمارين عملية 6
حلول مقترحة (تلميحات) 6
الفصل الثامن: الأداء 7
٨.١ التوازي عبر n_jobs 7
٨.٢ اختيار خوارزمية بحث معاملات أكفأ 7
٨.٣ تقليل استهلاك الذاكرة 7
أفضل الممارسات 7
الأخطاء الشائعة 8
الفصل التاسع: التصحيح 9
٩.١ رسائل الخطأ الشائعة 9
٩.٢ استراتيجية تشخيص عامة 9
٩.٣ فحص خطوات Pipeline بشكل منفصل 9
الأخطاء الشائعة 9
الأسئلة الشائعة 9
الفصل العاشر: أفضل الممارسات 10
١٠.١ أسلوب كتابة احترافي 10
١٠.٢ هيكلة مشروع تعلّم آلي احترافي 10
١٠.٣ الاختبار 10
١٠.٤ التحقق من تسرّب البيانات (Data Leakage Checklist) 10
أفضل الممارسات 10
الفصل الحادي عشر: مشاريع مصغّرة 12
مشروع ١: مقارنة شاملة لعدة نماذج مع تقرير نصي آلي 12
مشروع ٢: خط أنابيب تصنيف كامل ببيانات مختلطة وحفظ النموذج النهائي 12
مشروع ٣: أداة تحليل أهمية الخصائص (Feature Importance) 13

تمارين عملية 13
حلول مقترحة (تلميحات) 13
مراجع وروابط رسمية 13
خاتمة Scikit-learn 14

الفصل السابع: الأمثلة العملية

٧.١ المستوى المبتدئ

مثال ١: خط أنابيب تصنيف بسيط من البداية للنهاية

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression()
model.fit(X_train_scaled, y_train)
print('Accuracy:', accuracy_score(y_test, model.predict(X_test_scaled)))
```

شرح الكود: هذا النمط — تقسيم، تطبيع، تدريب، تقييم — هو الهيكل الأساسي الذي يتكرر في كل مشروع تعلّم آلي تقريبًا في Scikit-learn، بغض النظر عن الخوارزمية المستخدمة.

٧.٢ المستوى المتوسط

مثال ٢: خط أنابيب موحّد بـ Pipeline بدل خطوات منفصلة

```
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression()),
])

pipe.fit(X_train, y_train)   # X_train الخام، بلا تطبيع يدوي مسبق
print('Accuracy:', pipe.score(X_test, y_test))
```

شرح الكود: نفس نتيجة المثال الأول تمامًا، لكن الكود أقصر وأكثر أمانًا ضد تسرّب البيانات لأن Pipeline يضمن تطبيق fit_transform على التدريب فقط وtransform فقط على الاختبار تلقائيًا.

٧.٣ المستوى الاحترافي

مثال ٣: Pipeline كامل ببيانات مختلطة وبحث شبكي عن أفضل معاملات

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

numeric_features = ['age', 'income']
categorical_features = ['city']

numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])
categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore')),
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipe, numeric_features),
    ('cat', categorical_pipe, categorical_features),
])

full_pipe = Pipeline([
    ('preprocess', preprocessor),
    ('model', RandomForestClassifier(random_state=42)),
])

param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [5, 10, None],
}

grid = GridSearchCV(full_pipe, param_grid, cv=5, scoring='f1_macro')
grid.fit(X_train_df, y_train)
print(grid.best_params_, grid.best_score_)
```

شرح الكود: هذا مثال احترافي كامل يجمع كل ما سبق: Pipeline متداخل لكل نوع عمود، ColumnTransformer لدمجها، Pipeline نهائي يضم المعالجة والنموذج معًا، وGridSearchCV يبحث عن أفضل معاملات باستخدام صيغة التسمية model__parameter للوصول إلى معاملات خطوة معيّنة داخل Pipeline متداخل.

٧.٤ مثال من مشروع حقيقي: تقييم عدة نماذج ومقارنتها بمقياس موحّد

```
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

candidates = {
    'LogisticRegression': LogisticRegression(),
    'RandomForest': RandomForestClassifier(random_state=42),
    'SVM': SVC(),
    'DecisionTree': DecisionTreeClassifier(random_state=42),
}
```

```
results = {}
for name, clf in candidates.items():
    pipe = Pipeline([('scaler', StandardScaler()), ('model', clf)])
    scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring='accuracy')
    results[name] = scores.mean()

for name, score in sorted(results.items(), key=lambda kv: kv[1], reverse=True):
    print(f'{name}: {score:.4f}')
```

شرح الكود: بفضل الاتساق التام في Estimator API، يمكن وضع أي عدد من النماذج المختلفة تمامًا في قاموس واحد وتقييمها بنفس الحلقة البرمجية دون أي كود خاص لكل نموذج — وهذا التطبيق العملي الأوضح لفلسفة الفصل الثالث بأكملها.

تمارين عملية

١. ابنِ Pipeline يتضمن SimpleImputer وStandardScaler وSVC، ودرّبه على بيانات تحتوي قيمًا مفقودة.

٢. استخدم GridSearchCV للبحث عن أفضل عمق (max_depth) لشجرة قرار DecisionTreeClassifier عبر ٥ قيم مختلفة.

٣. قارن بين ٣ نماذج تصنيف مختلفة على نفس البيانات باستخدام cross_val_score وحلقة for.

حلول مقترحة (تلميحات)

١. Pipeline([('imputer', SimpleImputer()), ('scaler', StandardScaler()), ('model', SVC())])

٢. GridSearchCV(DecisionTreeClassifier(), {'max_depth':[2,4,6,8,None]}, cv=5)

٣. اتبع نمط قاموس candidates في المثال الأخير.

الفصل الثامن: الأداء

٨.١ التوازي عبر n_jobs

العديد من Estimators وأدوات اختيار النموذج في Scikit-learn تدعم معامل n_jobs للتوازي عبر أنوية المعالج المتعددة، وهو أسهل وأسرع طريقة لتحسين الأداء دون تغيير أي منطق.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=500, n_jobs=-1)   # -1 يعني استخدام كل الأنوية المتاحة

grid = GridSearchCV(model, param_grid, cv=5, n_jobs=-1)   # توازي على مستوى البحث الشبكي أيضًا
```

٨.٢ اختيار خوارزمية بحث معاملات أكفأ

الأداة	الاستراتيجية	متى تستخدمها
GridSearchCV	بحث شامل (Exhaustive) عبر كل التوافيق الممكنة	مساحة بحث صغيرة
RandomizedSearchCV	عيّنة عشوائية من التوافيق بعدد محدد مسبقًا	مساحة بحث كبيرة جدًا
HalvingGridSearchCV	استبعاد تدريجي للمرشحين الأضعف أداءً مبكرًا	موارد حسابية محدودة مع مساحة بحث كبيرة

```
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(model, param_distributions=param_grid, n_iter=20, cv=5, n_jobs=-1)
```

٨.٣ تقليل استهلاك الذاكرة

- استخدم dtype=np.float32 بدل float64 عند إدخال بيانات ضخمة، فمعظم Estimators تدعم كلا النوعين.
- احذف الأعمدة غير الضرورية قبل التدريب بدل الاعتماد على النموذج لتجاهلها ضمنيًا.
- استخدم partial_fit في الخوارزميات التي تدعمه (مثل SGDClassifier) للتدريب التدريجي على بيانات أكبر من الذاكرة.

```
from sklearn.linear_model import SGDClassifier

model = SGDClassifier()
for X_batch, y_batch in data_stream:
    model.partial_fit(X_batch, y_batch, classes=[0, 1])   # تدريب تدريجي دون تحميل كل البيانات دفعة واحدة
```

أفضل الممارسات

- فعّل n_jobs=-1 في النماذج والأدوات التي تدعمها لاستغلال كل أنوية المعالج.

- استخدم RandomizedSearchCV بدل GridSearchCV عند اتساع مساحة البحث عن المعاملات.
- فكّر في partial_fit للبيانات الأكبر من الذاكرة بدل محاولة تحميلها كاملة.

الأخطاء الشائعة

- استخدام GridSearchCV على مساحة بحث ضخمة جدًا (مئات التوافيق × ١٠ طيّات) دون تقدير الوقت المطلوب مسبقًا.
- نسيان n_jobs=-1 في نماذج تدعمها، مما يهدر إمكانات المعالج متعدد الأنوية بالكامل.

الفصل التاسع: التصحيح

٩.١ رسائل الخطأ الشائعة

رسالة الخطأ	السبب الشائع
ValueError: Found input variables with inconsistent numbers of samples	عدم تطابق عدد الصفوف بين X وy
NotFittedError: This Estimator is not fitted yet	استدعاء predict أو transform قبل fit
ValueError: could not convert string to float	تمرير عمود نصي/فئوي دون ترميز مسبق (OneHotEncoder مثلاً)
ValueError: Input X contains NaN	قيم مفقودة لم تُعالَج قبل التدريب في نموذج لا يدعمها أصلاً
ConvergenceWarning	النموذج لم يصل للتقارب ضمن max_iter المحدد، غالبًا بسبب بيانات غير مُطبَّعة

٩.٢ استراتيجية تشخيص عامة

```
print(X_train.shape, y_train.shape)   # التأكد من توافق الأبعاد أولاً دائمًا
print(X_train.dtypes if hasattr(X_train, 'dtypes') else X_train.dtype)
print(pd.isna(X_train).sum() if hasattr(X_train, 'isna') else
np.isnan(X_train).sum())
```

٩.٣ فحص خطوات Pipeline بشكل منفصل

```
# الوصول لخطوة معيّنة داخل Pipeline للتشخيص
print(pipe.named_steps['scaler'].mean_)      # القيم المتعلَّمة فعليًا من هذه الخطوة

# تنفيذ خطوات جزئية يدويًا للتأكد من شكل البيانات في كل مرحلة
X_step1 = pipe.named_steps['preprocess'].fit_transform(X_train)
print('shape after preprocessing:', X_step1.shape)
```

عند فشل Pipeline طويل، أفضل استراتيجية هي الوصول لكل خطوة عبر named_steps وتنفيذها يدويًا خطوة بخطوة لتحديد أين تحديدًا تتغيّر البيانات بشكل غير متوقع، تمامًا كما فعلنا مع سلاسل Pandas الطويلة سابقًا.

الأخطاء الشائعة

- استدعاء predict على Pipeline لم يُدرَّب بعد (نسيان fit)، مما يرفع NotFittedError واضحًا لكن يُربك المبتدئين أحيانًا.
- تمرير أعمدة نصية خام مباشرة لنموذج يتوقع مدخلات عددية بحتة دون ترميز مسبق.

الأسئلة الشائعة

س: كيف أعرف أي أعمدة سبّبت خطأ 'could not convert string to float' في Pipeline طويل؟

ج: افحص X_train.dtypes (في Pandas) قبل التدريب، وتأكد أن كل عمود نصي مُمرَّر عبر OneHotEncoder أو ما يعادله ضمن ColumnTransformer.

الفصل العاشر: أفضل الممارسات

١٠.١ أسلوب كتابة احترافي

- استخدم Pipeline دائمًا بدل خطوات منفصلة يدوية، حتى في التجارب السريعة، لتفادي تسرّب البيانات من العادة الأولى.
- ثبّت random_state في كل مكان يقبله (train_test_split، النماذج، GridSearchCV) لضمان نتائج قابلة لإعادة الإنتاج.
- افصل تعريف Pipeline عن كود التدريب والتقييم في دوال أو وحدات منفصلة قابلة للاختبار.
- وثّق افتراضات كل عمود (نطاق القيم المتوقع، وجود قيم مفقودة من عدمه) في تعليق أو docstring بجانب ColumnTransformer.

١٠.٢ هيكلة مشروع تعلّم آلي احترافي

```
project/
├── data/
├── src/
│   ├── features.py      # دوال بناء ColumnTransformer
│   ├── models.py         # تعريف Pipeline (نموذج + معالجة كامل)
│   ├── train.py          # سكربت التدريب الرئيسي
│   └── evaluate.py       # حساب المقاييس وحفظ التقرير
├── models/               # ملفات .joblib محفوظة
└── tests/
    └── test_pipeline.py
```

١٠.٣ الاختبار

```
def test_pipeline_predicts_expected_shape():
    pipe = build_pipeline()
    pipe.fit(X_sample, y_sample)
    preds = pipe.predict(X_sample)
    assert preds.shape[0] == X_sample.shape[0]

def test_no_data_leakage_in_scaler():
    pipe = build_pipeline()
    pipe.fit(X_train, y_train)
    # تعلّمت من التدريب فقط وليس الاختبار scaler التحقق أن معاملات
    assert pipe.named_steps['scaler'].n_samples_seen_ == len(X_train)
```

١٠.٤ التحقق من تسرّب البيانات (Data Leakage Checklist)

- هل تم استدعاء fit أو fit_transform على أي شيء غير بيانات التدريب فقط؟
- هل تم اختيار الخصائص (Feature Selection) قبل تقسيم train/test بدل بعده أو داخل Pipeline؟
- هل تُستخدم بيانات مستقبلية زمنيًا (Look-ahead) في مهام السلاسل الزمنية عن طريق الخطأ؟

أفضل الممارسات

- استخدم Pipeline دائمًا كعادة افتراضية، حتى للتجارب السريعة.

- ثبّت random_state في كل مكان لضمان قابلية إعادة الإنتاج.
- راجع قائمة تحقق تسرّب البيانات قبل تسليم أي نموذج للإنتاج.

الفصل الحادي عشر: مشاريع مصغّرة

مشروع ١: مقارنة شاملة لعدة نماذج مع تقرير نصي آلي

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=1)

def evaluate_models(models: dict, X, y) -> pd.DataFrame:
    rows = []
    for name, clf in models.items():
        pipe = Pipeline([('scaler', StandardScaler()), ('model', clf)])
        scores = cross_val_score(pipe, X, y, cv=5, scoring='f1')
        rows.append({'model': name, 'f1_mean': scores.mean(), 'f1_std':
scores.std()})
    return pd.DataFrame(rows).sort_values('f1_mean', ascending=False)

report = evaluate_models({
    'LogisticRegression': LogisticRegression(max_iter=1000),
    'RandomForest': RandomForestClassifier(random_state=1),
    'SVM': SVC(),
}, X_train, y_train)
print(report)
```

يطبّق هذا المشروع: دالة تقييم عامة قابلة لإعادة الاستخدام مع أي قاموس نماذج، تُنتج تقريرًا منظمًا كـ DataFrame — دمج مباشر بين Pandas وScikit-learn في سياق عملي واقعي.

مشروع ٢: خط أنابيب تصنيف كامل ببيانات مختلطة وحفظ النموذج النهائي

```
df = pd.DataFrame({
    'age': [25, 40, None, 35, 50],
    'income': [3000, 5000, 4500, None, 8000],
    'city': ['Cairo', 'Giza', 'Cairo', 'Alex', 'Giza'],
    'target': [0, 1, 0, 1, 1],
})

X, y = df.drop(columns='target'), df['target']

preprocessor = ColumnTransformer([
    ('num', Pipeline([('imputer', SimpleImputer(strategy='median')), ('scaler',
StandardScaler())]), ['age','income']),
    ('cat', Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
('encoder', OneHotEncoder(handle_unknown='ignore'))]), ['city']),
])

final_pipe = Pipeline([('preprocess', preprocessor), ('model',
LogisticRegression())])
final_pipe.fit(X, y)
```

```
import joblib
joblib.dump(final_pipe, 'final_model.joblib')
```

يطبّق هذا المشروع: خط أنابيب متكامل من بيانات خام فوضوية (قيم مفقودة، أعمدة مختلطة) وصولاً لنموذج مدرَّب ومحفوظ، جاهز للاستخدام مباشرة على بيانات جديدة خام دون أي معالجة يدوية إضافية عند التحميل.

مشروع ٣: أداة تحليل أهمية الخصائص (Feature Importance)

```
rf = RandomForestClassifier(random_state=1)
rf.fit(X_train, y_train)

importance_df = pd.DataFrame({
    'feature': [f'feature_{i}' for i in range(X_train.shape[1])],
    'importance': rf.feature_importances_,
}).sort_values('importance', ascending=False)

print(importance_df.head(10))
```

يطبّق هذا المشروع: قراءة خاصية feature_importances_ المُتعلَّمة (تنتهي بشرطة سفلية وفق الاصطلاح الذي شرحناه)، وتحويلها إلى DataFrame مرتّب عبر Pandas لتحليل أي الخصائص أكثر تأثيرًا في قرار النموذج — دون الخوض في كيفية حساب هذه الأهمية داخليًا.

تمارين عملية

١. وسّع مشروع ١ ليشمل نموذجًا رابعًا (DecisionTreeClassifier) في المقارنة.

٢. حمّل النموذج المحفوظ في مشروع ٢ عبر joblib.load وطبّقه على صف بيانات جديد واحد.

٣. ارسم أهمية الخصائص من مشروع ٣ كرسم أعمدة أفقي باستخدام Matplotlib.

حلول مقترحة (تلميحات)

١. أضف 'DecisionTree': DecisionTreeClassifier(random_state=1) إلى القاموس الممرَّر لـ evaluate_models.

٢. loaded = joblib.load('final_model.joblib'); loaded.predict(new_row_df)

٣. importance_df.plot(kind='barh', x='feature', y='importance') عبر واجهة Pandas/Matplotlib المتكاملة.

مراجع وروابط رسمية

- التوثيق الرسمي: https://scikit-learn.org/stable/
- دليل الأداء: https://scikit-learn.org/stable/computing.html
- الكود المصدري: https://github.com/scikit-learn/scikit-learn

خاتمة Scikit-learn

بهذا نُتم مرجعًا احترافيًا كاملاً لواجهة Scikit-learn البرمجية، من فلسفة الاتساق حتى خطوط الأنابيب الكاملة والبحث عن أفضل المعاملات. المكتبة الأخيرة في هذه السلسلة هي Jupyter Notebook، البيئة التي تجمع كل المكتبات السابقة في مساحة عمل تفاعلية واحدة.